

API Documentation - Manhattan

Endpoint WebSocket/STOMP

Method	Endpoint	Fungsi
WebSocket CONNECT	/ws	Membuka koneksi STOMP native WebSocket.
WebSocket CONNECT	/ws-sockjs	Endpoint fallback SockJS untuk browser lama.
STOMP SEND	/app/room.create	Membuat room, membuat session creator, broadcast USER_JOINED.
STOMP SEND	/app/room.join	Join room, verifikasi password, reset/record rate limit, broadcast participant.
STOMP SEND	/app/room.info	Mengambil info room: password, jumlah peserta, status aktif.
STOMP SEND	/app/message.send	Relay ciphertext dan IV ke topic room.
STOMP SEND	/app/key.exchange	Forward encrypted AES key ke private queue target.
STOMP SUBSCRIBE	/topic/room/{roomName}	Menerima pesan terenkripsi room.
STOMP SUBSCRIBE	/topic/room/{roomName}/events	Menerima event room dan daftar peserta.
STOMP SUBSCRIBE	/user/queue/private	Menerima response private, error, dan AES key exchange.

Endpoint Detail: /app/room.join

Method: STOMP SEND. Request: roomName, password, rsaPublicKey, displayName. Response sukses: ROOM_JOINED ke /user/queue/private. DTO: RoomJoinResult. Service: RoomService.joinRoom(), RateLimitService, SessionService, KeyExchangeService.

Screenshot Kode

API Evidence - /app/room.join

```
88         Map<String, Object> successResponse = new HashMap<>();
89         successResponse.put("type", "ROOM_CREATED");
90         successResponse.put("roomName", roomName);
91         successResponse.put("clientId", clientId);
92         log.debug("Sending ROOM_CREATED response to session {}", sessionId);
93         sendToUser(sessionId, successResponse);
94         log.debug("ROOM_CREATED response sent successfully");
95
96         // Broadcast USER_JOINED event (the creator is the first participant)
97         String rsaPublicKey = (String) payload.get("rsaPublicKey");
98         if (rsaPublicKey != null) {
99             keyExchangeService.broadcastPublicKey(roomName, clientId, rsaPublicKey, displayName);
100         }
101
102         // Broadcast updated participant list
103         broadcastParticipantList(roomName);
104
105         log.info("Room '{}' created by IP {}", roomName, clientId);
106
107     } catch (Exception e) {
108         log.error("Unexpected error in createRoom for room '{}': {}", roomName, e.getMessage(), e);
109         if (sessionId != null) {
110             sendError(sessionId, "INTERNAL_ERROR", "An unexpected error occurred: " + e.getMessage());
111         }
112     }
113 }
114
115 /**
116  * Handles room join requests.
117  * Receives: { roomName, password }
118  * On success: joins room, creates session, broadcasts USER_JOINED event.
119  * On failure: sends error to user's private queue.
120  */
121 @MessageMapping("/room.join")
122 public void joinRoom(Map<String, Object> payload, SimpMessageHeaderAccessor headerAccessor) {
123     String clientId = getClientId(headerAccessor);
124     String sessionId = headerAccessor.getSessionId();
125
126     if (clientId == null || sessionId == null) {
127         return;
128     }
129
130     String roomName = (String) payload.get("roomName");
131     String password = (String) payload.get("password");
132     String displayName = (String) payload.get("displayName");
133     if (displayName == null || displayName.isBlank()) displayName = "User_" + clientId;
134
135     log.debug("Room join request from IP {} for room '{}'", clientId, roomName);
136
137     // Check rate limiting
138     if (rateLimitService.isLocked(clientId, roomName)) {
139         long remaining = rateLimitService.getRemainingLockoutSeconds(clientId, roomName);
140         sendError(sessionId, "RATE_LIMITED",
```

Controller Evidence - MessageController

```
25     private final SessionService sessionService;
26     private final SimpMessagingTemplate messagingTemplate;
27
28     public MessageController(MessageRelayService messageRelayService,
29                             SessionService sessionService,
30                             SimpMessagingTemplate messagingTemplate) {
31         this.messageRelayService = messageRelayService;
32         this.sessionService = sessionService;
33         this.messagingTemplate = messagingTemplate;
34     }
35
36     /**
37      * Handles encrypted message send requests.
38      * Receives: { roomName, ciphertext, iv }
39      * Relays the ciphertext to all other connected clients in the room.
40      */
41     @PostMapping("/message.send")
42     public void sendMessage(Map<String, Object> payload, SimpMessageHeaderAccessor headerAccessor) {
43         String clientId = getClientIp(headerAccessor);
44         String sessionId = headerAccessor.getSessionId();
45
46         if (clientId == null || sessionId == null) {
47             return;
48         }
49
50         String roomName = (String) payload.get("roomName");
51         String ciphertext = (String) payload.get("ciphertext");
52         String iv = (String) payload.get("iv");
53
54         if (roomName == null || ciphertext == null || iv == null) {
55             sendError(sessionId, "INVALID_MESSAGE", "Missing required fields: roomName, ciphertext, iv.");
56             return;
57         }
58
59         // Verify the sender has an active session
60         if (!sessionService.hasActiveSession(clientId)) {
61             sendError(sessionId, "NO_SESSION", "No active session found. Please join a room first.");
62             return;
63         }
64
65         // Update last activity timestamp
```